

Depth, Oversmoothing, and Architecture: A Comparative Study of Graph Neural Networks on the Cora Citation Network

CSE 705 Deep Learning
McMaster University

Kiarash Ara
arak@mcmaster.ca

Yiheng Wu
wu1261@mcmaster.ca

2026-08-03

Abstract

Graph neural networks (GNNs) built from iterative message passing lose the ability to discriminate between nodes as depth increases, a phenomenon known as oversmoothing; this limits the receptive field available to applications such as recommendation on user-item graphs, community detection in social networks, and molecular property prediction, where relevant structure can lie several hops away. We compare four message-passing architectures: GCN, GraphSAGE, GAT, and GCNII, on the Cora citation network across depths 2, 4, 8, 16, and 32, using Dirichlet energy and mean average distance (MAD) as two complementary oversmoothing diagnostics. We capture both quantities at initialization, at the best-validation checkpoint, and at the final training epoch, which lets us separate a collapse attributable to the propagation operator from one attributable to training. Test accuracy degrades sharply beyond depth 4 for GCN, GraphSAGE, and GAT, but through distinct mechanisms rather than one shared failure: GraphSAGE and GAT do not train at all at depth 32, while GCN partially trains and develops a signature of early-layer energy collapse paired with late-layer growth. GCNII, whose identity mapping and initial-residual connection are designed against this dynamic, is the only architecture whose accuracy improves with depth. Among four tested mitigations, Jumping Knowledge restores the most deep-network performance (73.1% test accuracy at depth 32, against a 24.1% unmitigated baseline), while residual connections alone do not clearly outperform the unmitigated baseline at this depth.

Table of Contents

1	Introduction	4
2	Problem Statement, Review, and Background	4
3	Theory and Datasets	4
4	Implementation Details	4
4.1	Environment and Reproducibility	4
4.2	Repository Structure and Module Boundaries	5
4.3	Data Pipeline	5
4.4	Model Implementation	6
4.5	Mitigation Implementations	7
4.6	Training and Evaluation Harness	8
4.7	Metrics Implementation	9
4.8	Experiment Grid and Sweep Infrastructure	11
4.9	Visualization and Aggregation	12
4.10	Deviations from Published Setups, Consolidated	13
4.11	Placement Decision to Settle	14
5	Explanation of the Source Code	14
5.1	Repository Map and Reading Order	15
5.2	The Interface Contract in Code	15
5.3	Per-Module Walkthroughs	16
5.4	Testing	17
5.5	What the Code Does Not Do	18
6	Results and Discussion	18
6.1	Reporting Conventions	18
6.2	Baseline Reproduction	19
6.3	Depth Sweep: Classification Performance	20
6.4	Structural Collapse at Initialization	22
6.5	The Trained State, and What It Does to the Collapse Signature	23
6.6	Separating Oversmoothing from Optimization Failure	26
6.7	The Architecture That Does Not Degrade	27
6.8	Mitigation Ablation	28
6.9	Qualitative Embedding Analysis	29
6.10	Threats to Validity and Limitations	30
6.11	Synthesis	31
7	Recommendations for Future Work	32
7.1	Questions This Study Raised and Did Not Answer	32
7.2	Instrumentation Gaps	32
7.3	Extensions to the Experimental Design	33
7.4	Scale and Noise	33
7.5	Transfer to Physics-Informed Graph Learning	33
	References	34

1 Introduction

[PLACEHOLDER: Section 1, per docs/report_sections_1-3_outline.docx: motivation and problem domain, the semi-supervised node classification task, why depth is the axis of study, thesis and scope, contributions, report roadmap. Not drafted in this pass.]

2 Problem Statement, Review, and Background

[PLACEHOLDER: Section 2, per docs/report_sections_1-3_outline.docx: problem statement, oversmoothing in the literature, trainability at depth, mitigation approaches, large-scale and noisy graphs, the gap this study addresses. Not drafted in this pass.]

3 Theory and Datasets

[PLACEHOLDER: Section 3, per docs/report_sections_1-3_outline.docx: notation and graph foundations, the normalized Laplacian and its null space, message passing, architectures, mitigation mechanisms, Dirichlet energy and MAD, classification metrics, the Cora dataset and split. Not drafted in this pass.]

Placement note carried from Section 4.11 below: this section is the intended home for the *theory* of the diagnostic apparatus (the three capture points, the comparable band, the contraction slope). Section 4 describes only how that theory is implemented.]

4 Implementation Details

This section describes what was built and why, at the level of design decisions rather than line by line; Section 5 describes how the code expresses those decisions. Every claim below traces to a spec file (`spec/*_spec.md`) or to `DECISIONS.md`; where the two disagree, `DECISIONS.md` is the source that was followed, since it is the project-wide record and the specs are per-component snapshots that can lag it.

4.1 Environment and Reproducibility

Every run in the sweep records its own environment at write time rather than this section stating it once from memory. Reading a representative record (`results/gcn_none_d2_s0.json`) gives Python 3.14.4, PyTorch 2.13.0 (CPU build), PyTorch Geometric 2.8.0, device `cpu`. Every other record in `results/`, `results/fidelity/`, and `results/hpsearch/` carries the same fields in its own `environment` block.

Seeding uses the literal integer list 0 through 9, not ten values drawn from a meta-random-number generator, so a reproduction attempt has one fewer piece of state to recover. `SetSeed` seeds `torch`, `random`, and `numpy` together and is called twice on the path from configuration to trained model: once in `experiments/runner.py`’s `RunOne`, before the model is constructed, and again at the top of `train/harness.py`’s `TrainRun`. The first call is not redundant: weight initialization happens at model-construction time, which runs strictly before `TrainRun`’s own seeding, so a run seeded only inside `TrainRun` would draw its initial weights from whatever random state happened to be ambient at the call site rather than from the configured seed. Without the earlier call, two runs sharing

`config.seed` would produce different initial weights, which would break both the determinism the test suite asserts and the reproducibility claim the README makes.

The reproducibility path end to end, since the course grades explicitly on whether the submitted code reproduces the report’s results using the README’s instructions, and this section and the README must describe the same path: `data.LoadCora` downloads and caches Cora under `data/Cora/` on first use; `experiments.BuildGrid(arm)` enumerates one of six arms as pure `RunConfig` objects with no side effects; `experiments.RunSweep` iterates a grid, skips any configuration whose result file already exists (idempotent), and writes one JSON record per run to `results/`, `results/fidelity/`, or `results/hpsearch/` depending on the arm; `src/generate_report_figures.py` reads those records back through `viz.LoadRecords/BuildTable/Aggregate` and regenerates every figure under `figures/` and every table under `tables/`. Nothing in this path is interactive or order-dependent beyond running arm F before arms A-E, since F’s winning hyperparameter configuration is frozen into the module constants the other arms read.

4.2 Repository Structure and Module Boundaries

The repository is organized as `data/`, `models/`, `metrics/`, `mitigations/`, `train/`, `experiments/`, and `viz/` under `src/`, plus `results/`, `figures/`, `tables/`, `specs/`, and `tests/` at the project root. Each component is defined exactly once in its module and imported by whatever calls it; `notebooks/` (and the exploratory notebook `src/tests.ipynb`) is for exploration and figure generation only and is never the source of truth for a component’s implementation.

The interface contract is the shared surface that let the two-person team work in parallel, and is worth stating as an engineering decision rather than only documenting. Three clauses of it matter most: (1) every model’s `Forward` returns (`logits`, `layerEmbeddings`), where `layerEmbeddings` is a list of length `numLayers + 1` with index 0 the raw input; (2) the results record schema (one JSON object per run, described fully in Section 4.6 and Section 4.8) is the only channel `metrics`, `train`, and `viz` communicate through; (3) the PyG Data fields (`x`, `edge_index`, `y`, `train_mask/val_mask/test_mask`) keep their library names verbatim everywhere they are consumed. Because this surface was fixed before either side of the implementation was built, the harness side (architectures, training loop, sweep orchestration) and the metrics/visualization side could each be written, and tested, against a written contract rather than against each other’s in-progress code.

4.3 Data Pipeline

`data.LoadCora` loads the public-split Cora citation network through PyTorch Geometric’s Planetoid dataset and applies row-normalization (`torch_geometric.transforms.NormalizeFeatures`) as a fixed preprocessing step, not an ablation axis. The resulting `Data` object carries `x` of shape `[2708, 1433]` (float, each row summing to 1), `edge_index` of shape `[2, E]` (raw, undirected, no self-loops), `y` of shape `[2708]` (7 classes), and boolean `train_mask/val_mask/test_mask` of sizes 140/500/1000. `AssertGraphInvariants` checks every one of these properties and raises rather than warns if any fails, so a change to the loader or a PyG version upgrade that silently altered the graph’s shape would be caught immediately rather than propagating into a training run.

Row normalization is applied because Kipf and Welling’s published GCN setup (arXiv:1609.02907, Sec. 5.2) row-normalizes input features, and the reproduction claim in Section 6.1 requires matching that protocol. It is safe for the oversmoothing measurement for two separate reasons, both needed together: the same transform is applied to every run in the sweep, so it cannot confound any comparison the study makes (no run is ever compared against an unnormalized counterpart); and

the reported energy is a ratio (E_1/E_1), and per-node row scaling cancels in that ratio’s overall scale in the same way a global rescale does (Section 4.7 gives the exact argument).

Self-loops and the corresponding degree normalization are handled inside the convolution layers (GCNConv adds self-loops internally by default), not at the data level, so `data.LoadCora` ships the raw graph. `metrics` augments its own copy of the graph independently for the reasons given in Section 4.7, and asserts on construction that the `edge_index` it receives contains no self-loops, so a caller that accidentally passed an already-augmented graph would fail loudly rather than silently double-augmenting every degree.

4.4 Model Implementation

A single base class, `GnnModel`, owns the layer loop; four subclasses (`GcnModel`, `SageModel`, `GatModel`, `GcniiModel`) supply only the per-layer convolution. Mitigations attach by composition, not inheritance: they are constructed independently and injected at model construction as a list of `LayerHook` objects plus one `Readout` object, and `models` imports nothing from `mitigations`; both depend on two protocols declared inside `models` (`LayerHook.Apply(h, hPrev, edgeIndex) -> Tensor`, shape-preserving, called once per layer; `Readout.Apply(layerEmbeddings) -> Tensor`, called once after the loop). Inheritance was rejected because a mitigation is a modification applied to a model, not a kind of model: encoding residual, PairNorm, or Jumping Knowledge as an architecture subclass would need four mitigations times three architectures as classes before any combination is considered, and residual-plus-PairNorm together has no honest class name. Mitigation methods on the base class were rejected for a related reason: that moves the coupling up a level rather than removing it, since the base class would then have to know the full set of mitigations that exists and every new one would edit both the base class and its dispatch logic.

The unmitigated baseline is a null object, not a branch. An unmitigated model is constructed with `layerHooks = []` and `readout = LastLayerReadout()`, and the layer loop contains no “are there mitigations?” test anywhere: `Forward` calls `hook.Apply` for every hook in the (possibly empty) list and `readout.Apply` unconditionally. This is load-bearing rather than stylistic: because the baseline and every mitigated variant execute the identical code path, a measured difference between them is attributable to the mitigation itself rather than to a divergence in control flow, which is exactly the license the mitigation ablation (Section 6.7) needs to attribute its results to the mechanisms under test.

Every convolution is invoked with the uniform signature (`x`, `edgeIndex`, `x0`). GCNII’s update rule requires the fixed initial-residual term `x0` at every layer; GCN, GraphSAGE, and GAT ignore it. The cost of this uniformity is one unused argument for three of the four convolution types, accepted so that the layer loop contains no per-architecture branch on call signature.

GCNII resolves via uncounted input and output projections, so that `numLayers` stays a hop count comparable across all four architectures. PyG’s `GCN2Conv` requires its input and its initial-residual term at equal width, which none of the other three convolutions require, so `GcniiModel` cannot reuse the generic per-layer construction that lets `GcnModel/SageModel/GatModel` map `1433 -> hiddenDim` on layer 1 and `hiddenDim -> outDim` on the last layer inside the conv itself. `GcniiModel` instead bypasses `GnnModel.__init__` entirely (it calls `nn.Module.__init__` directly) and wraps exactly `numLayers` GCN2Conv hops at constant `hiddenDim` width, preceded by an uncounted `Linear(1433, hiddenDim)` input projection and, when the readout is `LastLayerReadout`, followed by an uncounted `Linear(hiddenDim, 7)` output projection that stands in for the final layer’s activation. Neither projection counts toward `numLayers`, and the projected input is never

written into `layerEmbeddings`: index 0 remains the raw input `X` for GCNII exactly as for every other architecture. This is load-bearing because depth is the study’s primary comparison axis: if GCNII’s nominal depth `L` corresponded to only `L - 1` real message-passing hops, its depth axis would not mean the same thing as GCN’s, GraphSAGE’s, or GAT’s, and every depth-matched comparison involving GCNII would be silently confounded. The known cost, stated plainly: GCNII carries two extra parameterized linear layers (`1433 -> 64` and `64 -> 7`) that the other three architectures do not, bounded in size and not scaling with depth, which can only overstate GCNII’s capacity relative to its peers, in GCNII’s favor, at every depth (revisited in Section 4.10 and Section 6.9).

The layer-embedding tap point is after the convolution, after all `layerHooks`, after the activation, and before dropout. Every headline figure in Section 6 is a curve over this exact tensor, so the tap point determines what those curves measure. After the hooks, because residual and PairNorm are the treatment under test: tapping before them would make a mitigated run produce the same energy curve as its unmitigated baseline, and the ablation would measure nothing. After the activation, because ReLU is the representation the next layer actually receives, and because applying it identically in every configuration means it cannot confound a comparison across architectures or depths, the same argument that makes uniform row-normalization safe in Section 4.7. Before dropout, for three independent reasons: dropout is stochastic, so a post-dropout tap would carry a mask-dependent noise floor with no relation to smoothing; dropout is inactive in eval mode, so identical weights would yield different recorded curves depending on train/eval state, a metric whose value depends on a mode flag the metrics consumer does not control; and inverted dropout rescales surviving units by $1/(1-p)$ at train time, which would inflate raw Dirichlet energy by a constant that the energy ratio absorbs but that MAD does not respond to identically, making the two metrics disagree for a reason that is an artifact rather than a finding.

`layerEmbeddings` is returned attached to the autograd graph; detaching happens at the capture site in `train`, not inside `models`. Under Jumping Knowledge the readout consumes the whole embedding stack to produce the logits, so the training loss backpropagates through every entry in the list. Detaching inside `Forward` would silently zero the gradient to every layer except the last, and JK would train incorrectly with no error raised anywhere.

4.5 Mitigation Implementations

Three mechanisms attach to the base model by the composition seam above; GCNII is not among them, since it is a fourth convolution type rather than a modification applied to one (Section 4.10 revisits the resulting terminology tension between the experimental sense of “mitigation” and the code-level sense of “architecture”).

Residual (`ResidualHook`) is plain identity addition, `h + hPrev`, with an explicit no-op when the two shapes disagree. Widths match only on interior layers: layer 1 maps `1433 -> hiddenDim` and, under `LastLayerReadout`, the last layer maps `hiddenDim -> outDim`, and neither admits an identity residual. No learned projection was added to reconcile the boundary layers, both because that would add parameters whose count depends on depth (confounding the residual arm’s capacity with the depth axis, the same objection that rules out first-layer-only weight decay in Section 4.6) and because it is not what Kipf and Welling’s Appendix B depth study does, which is the residual arm’s point of comparison. The skip is explicit and tested rather than incidental: a hook that silently returned `h` unchanged on any shape mismatch would let a misconfigured run report a residual arm whose residual never actually fired, with nothing erroring to reveal it.

PairNorm (`PairNormHook`) uses the scale-individual variant (PN-SI) at a fixed scale of 1.0: center

by subtracting the column mean over nodes, then divide each row by its own L2 norm and multiply by the scale, with the row-norm denominator clamped away from zero. The clamp exists because ReLU produces exact all-zero rows at depth, precisely the regime this study measures, and an unclamped division would produce `nan` there rather than a defined value. PN-SI was chosen over plain PN because the authors’ own reference implementation states that plain PN behaves badly under a symmetric normalized adjacency (their experiments used a row-normalized one). GCNConv is symmetrically normalized, which is exactly the case the authors flag. The scale is not tuned: it is data-dependent in the authors’ own usage, and the coordinated hyperparameter search (Section 4.8) holds one configuration fixed across every architecture and depth, so tuning PairNorm’s scale would compare a tuned mitigation against untuned baselines. This is stated as a limitation with its direction named: it can only understate PairNorm’s effectiveness, never overstate it.

Jumping Knowledge (`JkReadout`) aggregates `layerEmbeddings[1:]` by element-wise max pooling, not concatenation, and projects the result to `outDim` with a single `Linear(hiddenDim, outDim)` layer. This is load-bearing for the same reason the GCNII projections are: concatenation would make the readout’s input width, and therefore the JK arm’s parameter count, a function of depth ($T * \text{hiddenDim}$ for T pooled layers), so a JK accuracy curve against depth would confound the mitigation’s effect with a capacity increase at exactly the depths where the study claims the mitigation helps. Max pooling holds the readout’s input at `hiddenDim` regardless of depth. A grader familiar with the Jumping Knowledge paper may ask why the standard concatenation variant was not used; the answer is this capacity confound, and it is worth noting that the JK paper’s own citation-network experiments select depths from $\{2, 3\}$, so the concatenation variant was never exercised at the depths this study runs. Index 0 (the raw 1433-dimensional input) is excluded from the pool, since it cannot be max-pooled against `hiddenDim`-wide hidden states; the readout’s `FinalLayerIsLogits` property is `False`, so the final convolution under JK emits `hiddenDim` (not `outDim`), is activated and dropped out like every other layer, and the comparable band for a JK run is $1..L$ rather than $1..L-1$.

Hook ordering is residual before PairNorm, applied in list order by `experiments.runner._BuildLayerHooks` regardless of how the mitigation names were supplied. The order is not arbitrary: residual is part of the update rule (it defines what the layer’s output is), while PairNorm normalizes the resulting representation under an invariant (constant total pairwise squared distance across nodes). Applying PairNorm first and adding an un-normalized $H^2(1)$ afterward would leave the sum un-normalized and defeat that invariant.

4.6 Training and Evaluation Harness

Every run uses the Adam optimizer at the hyperparameter configuration the search in Section 4.8 selected (`learningRate=0.01`, `dropout=0.5`, `weightDecay=5e-4`), which coincides with Kipf and Welling’s published configuration. Weight decay is applied uniformly across every layer’s parameters rather than to the first layer only, which is what Kipf and Welling do. Their choice is reasonable at two layers and has no principled extension to 32, where “the first layer” would regularize roughly a thirtieth of the parameters, a materially different intervention wearing the same name. Uniform decay keeps the regularization from becoming an implicit function of depth, which the depth axis requires.

Early stopping and checkpoint selection are both driven by validation loss, and the two are different operations. Training halts when validation loss has not improved for `patience` consecutive epochs; the checkpoint that is reported and captured is the epoch of *minimum* validation loss, whose weights

are restored after the loop ends. Kipf and Welling’s own protocol specifies only the first of these (a stopping rule: maximum 200 epochs, a window of 10 in which training halts if validation loss does not decrease) and says nothing about which weights to keep once the loop has stopped. This study supplies the missing half by tying selection to the same signal that drives stopping, so `bestEpoch` is unambiguous. This is a deliberate deviation from the submitted proposal, which specified early stopping on validation *accuracy*: loss is used instead, for fidelity to the reference protocol the baseline-reproduction claim rests on, and because accuracy over 500 validation nodes moves in steps of 0.002, coarse enough that an accuracy plateau can mask a still-descending loss. Both quantities are recorded at every epoch in `trainingCurve`, so the accuracy-based alternative remains derivable from stored data rather than merely asserted to be immaterial.

Patience is generous (100 epochs) and constant across every architecture, depth, mitigation arm, and seed, with `maxEpochs=1000` as a ceiling that is never the thing that ends a converging run. This directly pre-empts the strongest available attack on Section 6.5’s central finding: a 32-layer network may sit on a plateau for many epochs before its loss begins to descend, or may never descend, and at Kipf and Welling’s own window of 10 such a run would halt around epoch 12 with a flat training curve indistinguishable from genuine untrainability, and the harness would have manufactured exactly the failure mode the study exists to diagnose. A generous, constant patience cannot manufacture a failure; it can only cost additional compute, and compute is not the binding constraint at Cora’s size. Holding patience constant across depth (rather than loosening it further at greater depth) is what keeps the depth comparison itself valid, for the same reason a per-depth learning rate would confound the comparison (Section 4.8).

Three metric captures are taken per run, each an explicit eval-mode forward pass under `torch.no_grad()`, never reused from a training-mode forward: epoch 0, before any gradient step; the selected checkpoint, after the loop ends and the best `state_dict` is restored; and the final epoch, on the weights the loop ended with. The capture order is **final, then restore, then checkpoint**: final-epoch metrics are computed *before* the checkpoint weights are restored, not after. Reversing this ordering would silently make `finalMetrics` a byte-for-byte duplicate of `checkpointMetrics` on every run, which would erase the one signal (Section 6.5) that distinguishes a run whose early stopping fired meaningfully from one it did not. `layerEmbeddings` are detached to CPU only at these three capture sites, never inside `models`, consistent with the attached-return decision above.

Macro-F1 is implemented in-repo as roughly ten lines of vectorized tensor operations (`train.MacroF1`), not imported from `scikit-learn`: a combined class-pair index is built from targets and predictions, `torch.bincount` turns it into a confusion matrix, and per-class precision, recall, and F1 are read off that matrix’s diagonal, column sums, and row sums, with a zero-denominator guard for any class absent from a batch. `scikit-learn` appears only in `requirements-dev.txt` and is exercised in exactly one test, which checks the in-repo implementation against `sklearn.metrics.f1_score` on fixed synthetic input covering an unrepresented class. This is worth stating explicitly because the course grades how much of the code the team wrote versus took from prior work, and an own implementation that is verified once against a reference is a materially different artifact from an imported call.

4.7 Metrics Implementation

`OversmoothingMetrics` is constructed once per run and holds the augmented graph $G_{\sim} = A + I$ as fixed state: the augmented edge index and $D_{\sim}^{-1/2}$ are computed once in the constructor and

reused by every subsequent per-layer call, rather than rebuilt per layer or per capture.

The metric graph is fixed to $\mathbf{G}^{\sim}$ for GCN, GraphSAGE, and GAT alike; it is not matched to whatever operator each architecture actually propagates with. This is load-bearing: the metric graph is a measuring instrument, not part of the model under test, and fixing it is what makes the embeddings the only thing that varies across the cross-architecture comparison. Matching the metric graph to GAT’s own propagation graph is ill-defined in the first place, since GAT’s attention re-weights the graph per layer and throughout training, so energy at consecutive layers would then be measured against different graphs, making the fitted contraction slope compare quantities with different units, and a model could lower measured energy purely by concentrating attention with representations otherwise unchanged, so the metric would partly measure itself. The stated cost, named rather than left implicit: for GAT, the fitted decay rate describes GAT’s embeddings with respect to Cora’s fixed graph structure, not with respect to GAT’s own attention-weighted propagation, legitimate and necessary for a cross-architecture comparison, but not the exact quantity Cai and Wang’s bound (arXiv:2006.13318) describes for an attention-based architecture.

Both Dirichlet energy and MAD are computed over the full 2708-node set, with no split mask applied to either. Oversmoothing is a claim about the learned representation, not about the labeled nodes, and neither metric consumes labels; restricting to `train_mask` would leave 140 of 2708 nodes, whose induced subgraph is close to edgeless, so an energy sum over it would be dominated by which few edges happened to survive rather than by smoothing.

Dirichlet energy is per-dimension ($E(H)$ divided by the layer’s width), which removes the effect of comparing a 1433-dimensional input against a 64-dimensional hidden state. It is computed as $E(H) = 0.5 * \sum_{(i,j) \text{ in augmented edges}} ||h_i/\sqrt{d_i} - h_j/\sqrt{d_j}||^2$, vectorized as a row-scale by `invSqrtDegree`, a gather of source and target rows by the augmented edge index, a squared-difference row-norm, and a single sum, with no Python loop over nodes or edges. Self-loop terms contribute exactly zero to this sum but change every node’s degree d_i , which is the entire reason the graph is augmented before the energy is computed rather than computed on the raw graph. MAD follows the reduction convention given directly in the source paper’s Eq. 1-4 (Chen, Lin, Li, Li, Zhou, and Sun, arXiv:1909.03211): a cosine-distance matrix, averaged over non-zero entries only at both the per-row stage and the final stage, with two implementation choices the paper leaves unstated but that the code makes explicit: the diagonal is excluded before reduction (inert for any row with positive norm, since a row’s self-cosine-similarity is always 1 and therefore already contributes a zero distance that the non-zero filter would drop regardless, but load-bearing for an all-zero row, where the norm clamp used elsewhere makes the self-similarity compute as 0 rather than the true 1, which would otherwise leave a spurious non-zero self-distance surviving the filter); and both reduction denominators are clamped to return exactly 0, not NaN, when a count is 0, which is exactly what the fully collapsed regime this study measures produces.

The comparable band, the layer indices that are metrically comparable to one another, is derived from tensor widths at capture time, never hardcoded to a fixed range. `SelectComparableBand` returns every index $l \geq 1$ whose width matches the modal width across `1..numLayers`, asserting the result is contiguous so an unanticipated architecture cannot silently produce a gapped band that the slope fit would fit across anyway. Index 0 is excluded unconditionally, since it is a different representation kind (sparse binary bag-of-words) regardless of its width, and mixing it into the band would confound aggregation-driven smoothing with the arbitrary scale and shape of the first learned projection. Under `LastLayerReadout` the final index is also excluded, since it is the 7-dimensional logit space: a decision space that cross-entropy actively contracts, so its energy tracks training quality rather than oversmoothing. This makes the band `1..numLayers - 1`. Under

Jumping Knowledge, the final convolution emits `hiddenDim` rather than `outDim` and is activated like every other layer, so index `numLayers` belongs in the band and the band is `1..numLayers`. This is load-bearing because a plotting function that assumed the `LastLayerReadout` band would silently truncate every Jumping Knowledge curve by one layer, exactly the failure the derived-band design exists to prevent, and it is checked directly in the test suite (Section 5.4).

Energies are floored at `1e-12` before the contraction slope is fitted by least squares on `log(energy)` against layer index, and the floor’s effect is quantified rather than left as an unstated numerical detail. A genuinely collapsed deep configuration produces per-dimension energy that is exact-zero in float32 at some band layers, and `log(0)` is undefined; flooring keeps the fit well-defined at the direct cost of capping how extreme the fitted slope can read once energy underflows the floor. For the unmitigated depth-32 GCN baseline (`gcn_none_d32_s0`), the floor is not a rare edge case: it binds on 6 of 31 band layers at epoch 0 (reported slope `-0.7579` against an unfloored `-0.8508`) and on 22 of 31 band layers at the checkpoint (reported slope `+0.8622` against an unfloored `+1.1435`). Both reported values understate the true magnitude of change, and the direction of the bias is named explicitly: the floor pulls a reported slope toward zero regardless of which direction it runs, so it can only make a collapsed configuration’s measured contraction (or growth) look shallower than it truly is, never manufacture an effect that is not present. `FitContractionSlope` returns `nan` rather than `0.0` when the band has fewer than two points, which is the `numLayers=2`, `LastLayerReadout` case; the two baselines at depth 2 have no fitted slope, by construction, and this must not be silently read as “zero contraction.”

The reported curve is not additionally divided by representation magnitude; $||H_1||_F^2$ (squared Frobenius norm) is recorded per layer instead, so a magnitude-normalized variant remains derivable at aggregation time without a rerun. Cai and Wang’s bound is $E_{\{1+1\}} \leq (1 - \lambda)^2 ||W||^2 E_1$, so weight-norm growth is inside the mechanism the bound describes, not noise sitting on top of it. A mitigation that restores deep performance partly by letting $||W||$ grow is succeeding by a route the theory anticipates, and dividing that growth out of the reported curve would hide exactly the effect Section 6.6 and Section 6.7 report.

4.8 Experiment Grid and Sweep Infrastructure

The sweep is a declarative grid (`experiments.BuildGrid`, a pure function with no side effects) plus a runner (`experiments.RunSweep`) that iterates it; no configuration logic lives inside the runner itself, so the grid can be enumerated and its size checked before anything executes. Six arms sum to 534 runs:

- Arm A: the unmitigated depth sweep: `{gcn, sage, gat}` x depth `{2,4,8,16,32}` x seed `0..9`. 150 runs.
- Arm B: the mitigation ablation on GCN: `{residual}`, `{pairnorm}`, `{jk}`, `{pairnorm, residual}`, x depth x seed. 200 runs.
- Arm C: GCNII across the same depth grid. 50 runs.
- Arm D: the mitigation arm B found most effective, carried to GraphSAGE and GAT: `{sage, gat}` x depth x seed, generated only after arm B is aggregated. 100 runs.
- Arm E: the fidelity arm, GCN, depth 2, `hiddenDim=16`, seed `0..9`, the published-baseline reproduction. 10 runs.
- Arm F: the hyperparameter search, GCN at depth 2, `learningRate` in `{0.005, 0.01}` x `dropout` in `{0.5, 0.6}` x `weightDecay` in `{5e-4, 1e-3}`, 3 seeds. 24 runs.

The depth grid `{2, 4, 8, 16, 32}` is log-spaced because Cai and Wang’s contraction bound

predicts exponential energy decay in depth, so log-spaced depths are evenly spaced on the log-energy axis every headline figure in Section 6 uses; a candidate additional depth of 24 was rejected for buying resolution only where the curve has already collapsed, at the cost of breaking that spacing.

The hyperparameter search (arm F) is tuned once, at depth 2 on GCN, and frozen across every architecture and depth in arms A through E. This is methodologically load-bearing rather than a shortcut: tuning per depth, or per architecture, would confound the depth effect the entire study measures with a hyperparameter effect, which is exactly the failure a per-group aggregation warning in the results schema exists to flag. Section 6.1 reports the search’s outcome and its honest limitation (three seeds do not cleanly separate the eight candidate configurations); the point made here is only that the search exists to justify one frozen configuration, not to find a global optimum.

The sweep is idempotent, with atomic writes and exactly one writer. `RunSweep` skips any configuration whose result file already exists unless explicitly forced, records are written to a temporary path and renamed into place so a file is either complete or entirely absent, and `experiments` is the only module that ever writes to `results/`. At 534 runs over a single-machine, multi-day window, an interrupted sweep restarting from zero each time is the failure mode this design exists to avoid; a failing individual run is caught, written as a `<runId>.failed.json` marker alongside its error and traceback, and the sweep continues, so a genuine failure remains visible and distinguishable from a configuration that was simply never scheduled, rather than silently aborting the remaining several hundred runs.

The results filename convention required a subdirectory split for arms E and F, found by the sweep’s own “no duplicate paths” test rather than assumed in advance. The base convention, `<convType>_<mitigations>_d<depth>_s<seed>.json`, does not encode `hiddenDim` or the training hyperparameters. Enumerating the full grid and computing every path showed only 500 distinct filenames where 534 were expected: arm E’s ten files are exact duplicates of ten of arm A’s (same `convType`, mitigations, depth, and seed, differing only in `hiddenDim`), and arm F’s 24 hyperparameter combinations collapse onto the three seed-only filenames arm A already claims. Run into one flat directory as an earlier draft of the convention specified, later arms would have silently overwritten earlier runs’ result files, a direct threat to every reported number, caught by hand arithmetic ($150 + 200 + 50 + 100 + 0 + 0 = 500$, not 534) before being treated as settled. The fix: arm E writes to `results/fidelity/` and arm F to `results/hpsearch/`, and arm F’s filenames additionally embed `learningRate/dropout/weightDecay` directly, since its 24 configurations still collide with each other inside `hpsearch/` alone (all share `gcn/none/depth 2`, varying only 3 seeds and 8 hyperparameter combinations). Neither change alters the 534-run total or arms A-D’s filenames.

Ten of the 534 configurations additionally persist their checkpoint-capture layer embeddings to `results/embeddings/*.pt`: GCN, seed 0, at depths 2 and 32, for each of the five mitigation arms (the unmitigated baseline plus arm B’s four combinations). These are existing arm A and arm B configurations with a flag set, not additional runs, and exist solely because the qualitative t-SNE analysis in Section 6.8 has no other source for the raw [2708, 64] embedding tensors; every other run writes only the reduced scalars `metrics` produces.

4.9 Visualization and Aggregation

Aggregation is strictly separated from plotting: `viz.LoadRecords` reads and validates every JSON record in a directory, `viz.BuildTable` flattens each record to one tidy row, and `viz.Aggregate` groups by a set of columns and returns mean, standard deviation, and count per group. Only after this does any `Plot*` function touch `matplotlib`. Mean and standard deviation are computed at

read time and never written back to `results/`, so the records directory remains the single source of truth with no derived file beside it that could go stale.

The comparable band is read from each record’s stored `bandIndices`, never recomputed by the plotting code. This mirrors the derivation rule in Section 4.7 from the consuming side: a plotting function that assumed the `LastLayerReadout` band (`1..L-1`) would silently truncate every Jumping Knowledge curve by exactly one layer, which is the failure this design choice exists to prevent, and it is checked directly in the viz test suite (Section 5.4).

The normalized energy curve plotted in Section 6, per-dimension energy divided by its value at the first band index, is formed by `viz.EnergyCurve` at plotting time, not stored in the record, so the reference layer could be changed without re-running the sweep. Values that come out `NaN` or infinite (a zero reference energy, for instance) are returned untouched rather than dropped, so a collapsed run shows a visible gap in a curve rather than a fabricated point.

t-SNE is the only embedding projection used; UMAP is not, since the proposal’s “t-SNE / UMAP” phrasing is read as an either/or rather than a requirement for both, and running two projections that could disagree slightly would raise a question with no analytical payoff. `PlotEmbeddingProjection` uses `sklearn.manifold.TSNE` with `random_state=0` and `perplexity=30`, both stated in the figure’s caption, and fits the projection separately for each panel: a shared fit across the shallow and deep panels would impose one neighborhood structure on both and manufacture the very difference the figure exists to illustrate.

Figure sizing was decided in advance rather than discovered during layout: every figure is 3.5 inches wide (matching a single IEEE column, for the article’s benefit) with font sizes set explicitly rather than inherited from `matplotlib`’s defaults, and rendered under the headless `Agg` backend.

4.10 Deviations from Published Setups, Consolidated

Each deviation below is stated once here, with its reason and the direction of its effect named, rather than left scattered across the sections above. A grader comparing this study against the original papers will find these regardless, and finding them already stated is different from finding them undisclosed.

- Hidden width 64, not each paper’s own published width. No single value matches every architecture’s publication (Kipf and Welling publish GCN at 16; Veličković et al. publish GAT at 8 heads of 8, total 64), and capacity parity across architectures is required for the depth comparison to mean one thing. At width 16 with GAT’s 8 heads, each head would attend over 2-dimensional keys, close to degenerate, placing a confound in the study’s headline figure unrelated to depth. The fidelity arm (Section 6.1) recovers the published width as a separately measured quantity rather than an unmeasured caveat.
- Uniform activation (ReLU) and uniform dropout (0.5) across all four architectures, where GAT publishes ELU and dropout 0.6. This is load-bearing rather than cosmetic: the post-activation tap-point argument in Section 4.4 depends on the same activation being applied everywhere, and ELU is not non-expansive with respect to Dirichlet energy the way ReLU is, so a mixed-activation study would attribute part of GAT’s energy curve to its nonlinearity rather than to its aggregation mechanism.
- Weight decay applied uniformly across all layers, rather than to the first layer only as Kipf and Welling do, for the reason given in Section 4.6.
- Early-stopping window (patience 100 versus 10) and epoch ceiling (1000 versus 200) loosened

relative to the published two-layer setup, for the reason given in Section 4.6: a tight window manufactures the depth-32 failure the study exists to diagnose.

- Checkpoint selection on validation loss where the submitted proposal specified validation accuracy, stated as a deliberate refinement rather than an unexplained change, with the accuracy-based alternative recorded in every run’s `trainingCurve` and therefore derivable from stored data rather than merely asserted to be immaterial.
- One shared hyperparameter configuration across four architectures whose published setups differ. GAT publishes learning rate 0.005 and dropout 0.6; GCNII publishes dropout 0.6 with a substantially larger convolutional weight decay than this study’s uniform $5e-4$. The direction is named: architectures whose published configuration differs most from the shared one are the ones most likely to be understated by it, GAT foremost among them.
- Mitigation configurations held at fixed, untuned values (PairNorm’s scale fixed at 1.0; Jumping Knowledge’s aggregation fixed to max pooling). This can only understate the mitigations’ effectiveness relative to a tuned variant, never overstate it.
- GCNII’s architecture hyperparameters (`alpha=0.1`, `theta=0.5`) come from its own published Cora configuration (Chen et al., arXiv:2007.02133, Table 6), while its training hyperparameters come from this study’s shared configuration rather than the paper’s own (`dropout=0.6` and a much larger convolutional weight decay in the original). This is a hybrid configuration, not a reproduction, and is named as such rather than left to imply a full match to the published setup.
- GCNII carries two uncounted projection layers that GCN, GraphSAGE, and GAT do not (Section 4.4), bounded in size and not scaling with depth, which can only overstate GCNII’s capacity relative to its peers, in GCNII’s favor, at every depth.

4.11 Placement Decision to Settle

The diagnostic apparatus this study reports through (the three metric capture points, the derived comparable band, and the fitted contraction slope) has been described here purely as an implementation (what is computed, and how). The theoretical grounding for *why* these particular quantities characterize oversmoothing (the normalized-Laplacian contraction bound, the null-space argument behind the comparable band’s exclusions) belongs to Section 3 once it is drafted. This is stated explicitly, as the outline this section was drafted from requires: Section 3 owns the theory, Section 4 owns the implementation of that theory, and Section 6 should cite the metric definitions from Section 3 rather than re-derive them.

5 Explanation of the Source Code

Drafting note, to be removed once resolved. This section is drafted directly from the source code and the specs, as the best available substitute for a defended walk-through. The outline this report follows states plainly that Section 5 should be written only for components that have passed a defense gate in the Claude.ai Project, since a subsection written about code nobody has walked through line by line is exactly the passage that cannot be defended on video or survive the course’s zero-tolerance plagiarism review. That gate history is not visible from this session. Before submission, each subsection below needs to be checked against the actual gate record and either confirmed or reworked.

This section is a walkthrough of how the code expresses the design Section 4 describes, at a level

where a reader with the repository open can follow along. It is not Section 4 restated and not a code listing. Detail is kept at the per-module level throughout: the key classes and functions in each module, the data flowing between them, and short excerpts only where an excerpt makes a point prose alone cannot.

5.1 Repository Map and Reading Order

The build order and the dependency order coincide, and are the order to read the modules in: `data`, then `models`, then `metrics`, then `train`, then `mitigations`, then `experiments`, then `viz`. `data` has no internal dependencies and is the root of the graph. `models` depends only on `data`'s field names and declares the two protocols `mitigations` will later implement, without importing `mitigations` itself. `metrics` depends on `data` and on the shape of `models`' output, but not on `train`, `mitigations`, or `experiments`: it is a fixed instrument, not a configuration-aware component. `train` depends on `models` and `metrics` and owns the only optimizer in the repository. `mitigations` depends only on the two protocols `models` declares, and sits after `train` in the build order specifically because the null-object default lets the harness run and the baselines reproduce before any mitigation exists to implement. `experiments` depends on every other module and is the only one that writes to disk. `viz` depends on `data` (for labels only) and on the records `experiments` writes, but is never imported by anything: it is the leaf of the dependency graph and the only module whose output a reader sees directly.

Three entry points sit outside this module graph: `src/run_sweep.py` executes the sweep against the six arms in the order F, A, B, C, E, D; `src/generate_report_figures.py` regenerates every figure and table from `results/` alone; and the pytest suite under `src/tests/` is the third, run directly rather than through either driver.

5.2 The Interface Contract in Code

`GnnModel.Forward(x, edgeIndex) -> tuple[Tensor, list[Tensor]]` (`src/models/gnn_model.py`) is the one place the model side of the contract is expressed: it returns (`logits`, `layerEmbeddings`), with `layerEmbeddings[0]` always the raw input tensor `x` and every subsequent entry the corresponding layer's post-activation, pre-dropout output.

The results record is assembled entirely inside `train.TrainRun` (`src/train/harness.py`) and written by exactly one caller, `experiments.RunSweep` (`src/experiments/runner.py`), via `_AtomicWriteJson`. `train` never touches the filesystem; `TrainRun` returns a plain dict, and a test can call it directly against an in-memory config with no filesystem fixture. This single-writer property is what keeps the filename convention (`experiments.runner.ResultPath`) in one place: nothing else in the repository constructs a `results/*.json` path independently, so the convention cannot drift between two call sites.

Two runtime assertions enforce contract clauses the type system cannot: in `experiments.runner.RunOne`, `("jk" in config.mitigations) == (config.readout == "jk")` is checked before training begins, so a `RunConfig` in which the mitigation list and the readout disagree about Jumping Knowledge fails immediately rather than producing a record with an internally inconsistent `config` block; and in `metrics.oversmoothing.BuildAugmentedOperator`, the incoming `edgeIndex` is checked for self-loops and the constructor raises if any are found, so a caller that passed an already-augmented graph could not silently double-augment every node's degree.

5.3 Per-Module Walkthroughs

data (`src/data/cora.py`). Owns `LoadCora(root, normalizeFeatures) -> Data` and `AssertGraphInvariants(data) -> None`. It does not own self-loop insertion or degree normalization; both are deliberately left to the consumers that need them. Every invariant check (`num_nodes`, feature shape, class count, undirectedness, self-loop absence, mask sizes, pairwise mask disjointness) is a single vectorized tensor reduction (`.sum()`, boolean `&`, PyG’s `contains_self_loops`); there is no Python loop over nodes anywhere in the module. The one non-obvious point: `AssertGraphInvariants` raises `ValueError` explicitly rather than using a bare `assert`, so the checks cannot be compiled out under `python -O`.

models (`src/models/gnn_model.py`, `architectures.py`, `protocols.py`). Owns the layer loop and the four architecture subclasses; does not own mitigation logic, which it only declares an interface for. `GnnModel.__init__` builds `self.convs` as an `nn.ModuleList` by calling the subclass-supplied `BuildLayerConv` once per layer, computing each layer’s input and output width from `numLayers`, `hiddenDim`, and whether the current layer is the final, logit-emitting one. `Forward` (reproduced in Section 4.4’s data-flow description) is nine lines excluding the docstring: a `for` loop over `self.convs`, applying each hook in `self.layerHooks` in list order, then conditionally activating and tapping into `layerEmbeddings`, then conditionally applying dropout. `GcniiModel.Forward` (in `architectures.py`) duplicates this loop rather than reusing the base class’s, since its input and output projections do not fit the base class’s per-layer construction: this is the one place in `models` where the same shape of loop is written twice, a direct consequence of `GCN2Conv`’s equal-width constraint. The non-obvious point worth a reader’s attention: `_GatConvAdapter` divides the requested total output width by the head count before constructing `GATConv`, so that `hiddenDim` denotes total width across heads rather than per-head width, keeping GAT’s capacity comparable to GCN’s and GraphSAGE’s at the same `hiddenDim` rather than inflated by the head count.

metrics (`src/metrics/oversmoothing.py`). Owns `OversmoothingMetrics`, `DirichletEnergy`, `MeanAverageDistance`, `SelectComparableBand`, and `FitContractionSlope`; does not own anything about training, mitigations, or experiment configuration: it consumes tensors and a graph, never a model or a config object. `DirichletEnergy` is one row-scale, one gather by edge index, one squared-difference row-norm, and one sum: `[2708, hiddenDim]` in, one Python `float` out, for every layer in a run. The numerical guards, and why each exists, are the load-bearing content of this module. `MeanAverageDistance` clamps the row-norm denominator to `1e-12` before normalizing, since ReLU produces exact all-zero rows at depth and an unclamped division would produce `nan` rather than a defined cosine distance; it separately clamps both of MAD’s own reduction denominators (Eq. 3’s per-row count and Eq. 4’s final count) to return `0.0` rather than `NaN` when a count is zero, which is what a fully collapsed representation (every row identical, every pairwise distance zero) produces. Without these guards, the metrics this study’s entire depth-32 result rests on would return `NaN` in precisely the regime the study is about, rather than the well-defined zero that regime should report. `FitContractionSlope` carries the third guard, the `1e-12` energy floor discussed in Section 4.7, needed because a genuinely collapsed configuration’s per-dimension energy underflows to exact-zero in float32 and `log(0)` is undefined.

train (`src/train/harness.py`). Owns the optimizer, the training loop, and result-record assembly; does not own the filename convention or any file I/O. `TrainRun` is the sole public entry point most other code calls; `EvaluateSplit`, `CaptureMetrics`, and `MacroF1` are its private-in-spirit helpers, reused directly by the per-epoch validation pass and by the three capture points. The training loop itself (the `for epoch in range(1, maxEpochs + 1)` block) is 20-odd lines: forward, cross-entropy

on `train_mask` rows, backward, optimizer step, then an eval-mode validation pass whose loss decides whether `bestState` is updated and whether the patience counter resets. Vectorization here is inherited rather than original to this module: the forward and backward passes vectorize because `models` and PyG’s message-passing kernels do, and `MacroF1`’s own vectorization (a `bincount` over a combined class-pair index) is described in Section 4.6.

`mitigations` (`src/mitigations/hooks.py`, `readouts.py`, `naming.py`). Owns three mechanism implementations and one naming helper; does not own `GnnModel`, `train`, or `experiments`, and imports only the two protocols `models` declares. `ResidualHook` and `PairNormHook` carry no learnable parameters and are plain Python objects rather than `nn.Module` subclasses; `JkReadout` does carry a `Linear` layer and is an `nn.Module`, since `GnnModel` stores `readout` as a direct attribute and PyTorch auto-registers it as a submodule, which is what lets that layer’s parameters reach the optimizer alongside every convolution’s. `JkReadout.Apply` asserts that every tensor in `layerEmbeddings[1:]` shares one width before pooling, so a misconfigured model would raise rather than silently broadcasting a shape mismatch into the pooling operation.

`experiments` (`src/experiments/grid.py`, `runner.py`). Owns the declarative grid, model construction from a `RunConfig`, and all filesystem writes; does not own any numerical computation. `BuildGrid` is six private `_BuildArm*` functions behind one dispatcher, each a small set of nested `for` loops over Python lists with no side effects: calling `BuildGrid` twice returns equal lists and touches no filesystem, which is what lets the grid’s size be asserted directly in the test suite. `RunOne`’s ordering (seed, then build the model, then call `TrainRun`) is the one place the seeding subtlety described in Section 4.1 is visible in code: `SetSeed(config.seed)` is called before `BuildModel`, one line before the model’s weights are drawn.

`viz` (`src/viz/aggregation.py`, `figures.py`, `tables.py`). Owns loading, tidying, aggregating, plotting, and table export; does not own any model construction and imports `data` for label colors only. `LoadRecords` validates every record’s top-level keys against the schema and raises on a malformed file rather than skipping it silently, so a partially written or corrupted record surfaces immediately rather than quietly shrinking a seed count. `BuildTable` and `Aggregate` are the two functions every `Plot*` function and every exported table pass through; no plotting function reads a raw JSON record directly except `PlotEnergyVsLayer` and `PlotLossCurves`, which need one specific run’s full per-layer or per-epoch array rather than a cross-seed aggregate.

5.4 Testing

The `pytest` suite under `src/tests/` replaces the “runs top to bottom without errors” guarantee a single assembled notebook would otherwise have to provide informally. Four categories of test recur across the seven test modules: numerical correctness against a hand-computable or externally verifiable reference (for instance, `metrics`’ dense-reference energy test, which checks the vectorized edge-list computation against a dense $\text{trace}(H^T * \text{Laplacian} * H)$ form on a small synthetic graph, and `train`’s macro-F1 test against `sklearn.metrics.f1_score`); grid enumeration and uniqueness (`experiments`’ grid-arithmetic and no-duplicate-paths tests); shape and contract assertions (`models`’ band-width-homogeneity and final-entry-identity tests, conditioned on the readout); and qualitative gates, which check a directional property on real, trained output rather than an exact value (the smoke test that a 2-layer GCN reaches roughly 81 percent test accuracy; the depth-32 collapse gate described next).

Two tests caught real defects before they reached a reported number, and both are worth naming as such rather than only as passing tests. The path-collision tests in `test_experiments.py`

(`test_arm_e_and_f_collide_within_a_flat_directory` and `test_arm_f_still_collides_within_its_own_s`) are what surfaced the filename collision between arms E/F and arm A described in Section 4.8. Without them, arm E’s ten fidelity runs and several of arm F’s hyperparameter-search runs would have silently overwritten arm A’s result files, corrupting the depth-sweep aggregate with no error anywhere. The collapse-floor tests in `test_metrics.py` (`test_collapse_floor_on_regular_graph` and `test_collapse_floor_on_cora_is_sqrt_degree_weighted`) are what surfaced that Cora’s actual null space is the `sqrt(degree)` direction rather than the literal constant vector, a property of the symmetric-normalized Laplacian on an irregular graph that an assumption of “identical rows collapse to zero energy” would have gotten wrong on Cora specifically, even though it holds on a regular graph.

One test is gated on the epoch-0 capture rather than the checkpoint capture, and the reason is itself a result worth stating here. `test_depth_qualitative_gate_at_epoch_zero` asserts that a 32-layer unmitigated GCN shows monotonically decreasing per-dimension energy at initialization, not after training: at the checkpoint, under this study’s settled hyperparameters, that monotonic property does not hold (Section 6.3 and Section 6.4 report why), so gating the test there would make it flaky or outright false under a correct implementation. This connects directly to Section 6.5’s central finding: separating “the propagation operator collapses representations” from “training collapses representations differently” is not only a results-section distinction, it is a distinction the test suite itself had to make to stay meaningful.

5.5 What the Code Does Not Do

No per-layer gradient norm and no per-layer weight norm is persisted anywhere in the results schema, for any run. `TrainRun` calls `.backward()` every training step but never captures or stores the resulting gradient magnitudes, and no run’s trained weights are checkpointed to disk once training completes. This is stated plainly here because Section 6 and Section 7 both depend on it: it is the reason the mechanism behind GCN’s depth-32 early-layer energy collapse (Section 6.4) remains an inferred explanation rather than a demonstrated one, and it is the single instrumentation gap Section 7.2 identifies as most directly closing that gap if added.

6 Results and Discussion

Every number in this section traces to a `FINDINGS.md` entry (F-001 through F-007) or to a table or figure regenerated from `results/*.json` by `src/generate_report_figures.py` on 2026-07-22, after the corrections described in F-001 and F-002 below. No number here is transcribed from a conversation, a summary, or a remembered run. Where `FINDINGS.md` marks a clause as measured, it is stated flat below; where it marks a clause as inferred, it is phrased as *consistent with*, not *because*. This distinction is preserved throughout this section rather than allowed to blur, since it is the single most attackable property of the results and the one the study has invested the most care in getting right.

6.1 Reporting Conventions

Every result below is reported as mean and standard deviation across 10 seeds (3 seeds for the hyperparameter search, Section 6.1), on the standard 140/500/1000 public Cora split, unless stated otherwise.

Accuracy and macro-F1 are summarized by arithmetic mean; the energy ratio is summarized

by geometric mean and range, and the two conventions are not interchangeable. Accuracy and macro-F1 are bounded, roughly symmetric quantities across seeds, for which an arithmetic mean is the ordinary and appropriate summary. The energy ratio E_{last}/E_1 is a multiplicative, log-scale quantity that spans many orders of magnitude across seeds (26 orders of magnitude at depth 32, Section 6.4). An arithmetic mean of such a quantity is dominated by whichever single seed happens to be largest, to the point that “the mean of 10 seeds” can be arithmetically close to “one seed’s value, divided by 10.” Geometric mean is the summary statistic appropriate to a log-scale quantity, and it is used for every energy-ratio figure and table below; where a range is also reported, it is the minimum-to-maximum span across the 10 seeds, not a standard deviation, since a standard deviation of a quantity spanning 26 orders of magnitude is not a meaningful number.

Every energy or MAD figure and table below names which of the three capture points it reads: epoch 0 (before any gradient step), the selected checkpoint (the weights the reported test accuracy comes from), or the final epoch (the weights the training loop ended with, before any restore). A number or curve with no named capture has implicit, and therefore unverifiable, provenance; this report does not present one.

6.2 Baseline Reproduction

The fidelity arm confirms the baseline-reproduction claim: `hiddenDim=16` and `hiddenDim=64` give statistically indistinguishable test accuracy at depth 2. Arm E (`hiddenDim=16`, matching Kipf and Welling’s published width) reaches mean test accuracy 81.60% (± 0.31); arm A’s depth-2 GCN subset (`hiddenDim=64`, the width used throughout the rest of this sweep) reaches 81.49% (± 0.32). The difference, 0.11 points, is well within one standard deviation of either arm and not distinguishable given the observed seed variance. Both land close to Kipf and Welling’s own published figure of approximately 81.5%.

Table 1: Depth-2 GCN test accuracy at the published width (`hiddenDim=16`, arm E) versus this study’s sweep width (`hiddenDim=64`, arm A). Ten seeds each.

convType	hiddenDim	numLayers	testAccu- racy_mean	testAccu- racy_std	count
gcn	16	2	0.8160	0.0031	10
gcn	64	2	0.8149	0.0032	10

This licenses every subsequent deviation from the published width: since the width change from 16 to 64 does not materially move the depth-2 baseline, the capacity-parity argument for using 64 throughout the sweep (Section 4.10) costs nothing measurable at the one depth where a published number exists to check it against.

The hyperparameter search (arm F) selects a configuration that coincides exactly with Kipf and Welling’s published one, but the search itself does not cleanly separate its eight candidates. Ranked by mean validation accuracy over 3 seeds:

Table 2: Arm F, all eight hyperparameter combinations, GCN at depth 2, ranked by mean validation accuracy over 3 seeds (F-007).

learningRate	dropout	weightDecay	valAccu- racy__mean	valAccuracy_std
0.01	0.5	0.0005	0.8047	0.0042
0.01	0.5	0.001	0.8000	0.0020
0.005	0.5	0.001	0.7993	0.0031
0.01	0.6	0.0005	0.7993	0.0031
0.01	0.6	0.001	0.7987	0.0023
0.005	0.6	0.0005	0.7980	0.0020
0.005	0.6	0.001	0.7967	0.0012
0.005	0.5	0.0005	0.7933	0.0042

The winner (`learningRate=0.01`, `dropout=0.5`, `weightDecay=0.0005`) wins on the primary selection criterion (highest mean validation accuracy) without needing the tie-break rule: the runner-up sits 0.0047 below it, only marginally outside the winner’s own one-seed-noise band of 0.0042. Read plainly, though, three seeds do not cleanly separate these eight configurations: all eight span a range of only 1.14 accuracy points, while individual per-configuration standard deviations (from only 3 seeds) run 0.12 to 0.42 points, comparable in scale to the total spread across all eight. What makes the winner a reasonable choice regardless of that closeness is that it is *also* the configuration closest to the published baseline, the search’s own final fallback criterion, so the practical selection does not depend on trusting a ranking this close to noise: the fallback criterion alone would have selected the same configuration even if the primary ranking were pure noise.

6.3 Depth Sweep: Classification Performance

Test accuracy and macro-F1 against depth, for the unmitigated arm A sweep (three architectures) and arm C (GCNII, discussed on its own terms in Section 6.6):

Table 3: Test accuracy and macro-F1 versus depth, unmitigated GCN/GraphSAGE/GAT, ten seeds each. Macro-F1 computed directly from `results/*.json`, not previously tabulated by `src/viz`.

convType	depth	testAccu- racy__mean	testAccu- racy__std	macroF1__mean	macroF1__std
gcn	2	0.8149	0.0032	0.8032	0.0032
gcn	4	0.7368	0.0656	0.7381	0.0495
gcn	8	0.3349	0.1162	0.2864	0.1013
gcn	16	0.2352	0.0351	0.2104	0.0309
gcn	32	0.2407	0.0160	0.2102	0.0161
sage	2	0.8010	0.0042	0.7917	0.0040
sage	4	0.7172	0.0463	0.7086	0.0479
sage	8	0.2722	0.1078	0.0902	0.0734
sage	16	0.2850	0.0717	0.0627	0.0135
sage	32	0.2265	0.0983	0.0514	0.0189
gat	2	0.7890	0.0157	0.7780	0.0153
gat	4	0.7450	0.0593	0.7391	0.0641

convType	depth	testAccu- racy_mean	testAccu- racy_std	macroF1_mean	macroF1_std
gat	8	0.3228	0.1005	0.1563	0.1540
gat	16	0.2845	0.0727	0.0626	0.0137
gat	32	0.1966	0.0846	0.0459	0.0160

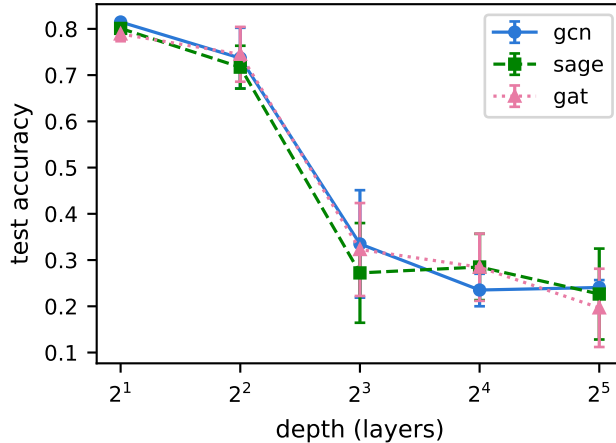


Figure 1: Test accuracy versus depth, unmitigated GCN, GraphSAGE, and GAT (arm A).

All three conventional architectures degrade with depth, and GCNII is the sole exception (Table 3, Figure 1; GCNII’s own curve is deferred to Section 6.6). The degradation is not gradual across the full grid: all three hold within a few points of their depth-2 accuracy at depth 4, then drop sharply between depths 4 and 8, and remain low through depth 32.

Macro-F1 collapses more sharply than accuracy for all three degrading architectures, most visibly for GraphSAGE and GAT. At depth 32, GAT’s accuracy (19.66%) is roughly 5 points above the uniform-random floor for seven classes (14.3%), while its macro-F1 (4.59%) is far below what either a uniform-random or majority-class predictor would produce on a balanced macro-averaged metric. This is *consistent with* predictions concentrating onto a small number of classes rather than spreading errors evenly across all seven. A per-class confusion matrix was not computed for this report, so that specific mechanism is not independently checked, only consistent with the size of the accuracy/macro-F1 gap.

The comparison against the majority-class floor is the sharpest single number in this depth sweep, and it holds for every seed, not only on average. The Cora test split’s majority-class floor, computed directly from the test-mask label counts ([130, 91, 144, 319, 149, 103, 64], summing to 1000), is 31.9%, not the approximately 30% figure sometimes quoted from memory. At depth 32, unmitigated GCN’s mean test accuracy (24.07%) sits below this floor, and its full 10-seed range (21.2%-26.2%) sits below it entirely: even GCN’s single best seed is 5.7 points short of a predictor that always guesses the majority class.

6.4 Structural Collapse at Initialization

At epoch 0 (before any gradient step, on the initialized weights only), GCN and GAT collapse cleanly and monotonically in both direction (MAD) and magnitude (energy ratio); GraphSAGE collapses in direction only. This is the initialized network as a whole: the propagation operator composed with each layer’s randomly initialized, untrained weight matrices, not the propagation operator in isolation: the random weights’ magnitudes are part of the composition and part of what drives the observed contraction, exactly as Cai and Wang’s bound has it ($||W||$ does not vanish merely because W is untrained).

Table 4: Epoch-0 (pre-training) MAD and energy ratio versus depth, unmitigated GCN/GraphSAGE/GAT (F-002).

convType	metric	depth 2	depth 4	depth 8	depth 16	depth 32
gcn	MAD	0.600	0.240	0.085	0.023	0.0045
gcn	energy ratio (E_{last}/E_1)	1.0	3.14e-2	3.14e-4	2.60e-7	6.22e-13
gat	MAD	0.596	0.211	0.076	0.017	0.0041
gat	energy ratio (E_{last}/E_1)	1.0	7.26e-2	2.73e-3	1.28e-5	1.25e-10
sage	MAD	0.083	0.0002	0.000	0.000	0.000
sage	energy ratio (E_{last}/E_1)	1.0	18.3	19.8	20.3	19.3

GCN’s energy ratio decays roughly 13 orders of magnitude by depth 32; GAT’s decays similarly. Both are *consistent with* the qualitative shape Cai and Wang’s bound predicts (repeated multiplicative contraction). This is explicitly not a claim of numerical agreement with the bound itself, since neither $||W||$ nor the graph’s spectral gap was computed and the measured decay rate was not checked against the bound’s right-hand side.

GraphSAGE’s directional collapse (MAD) and its energy ratio dissociate, and the dissociation is explained by a measured geometric fact rather than left as a coincidence. GraphSAGE’s MAD reaches near-zero already by depth 4, faster than either GCN or GAT, while its energy ratio *rises* from depth 2 to depth 4 and then plateaus around 19-20 at every depth tested, never decaying toward zero. Measured directly by singular-value decomposition at depth 16 (three seeds): GraphSAGE’s representation matrix has top-singular-value energy fraction 1.0000, an exact rank-1 matrix, not merely dominated by one direction, and its top singular vector matches the constant direction with cosine 1.0000, versus cosine 0.9512 to the `sqrt(degree)` direction. GCN and GAT, by contrast, are only mostly rank-1 (energy fraction 0.93-0.98) and do not cleanly separate the two candidate directions. This connects directly to a fixed geometric property of the metric graph itself (the augmented, symmetric-normalized Laplacian’s null space on Cora is the `sqrt(degree)` direction, not the literal constant vector, since Cora’s degree distribution, 2 to 169, is far from regular): GraphSAGE collapses exactly onto the constant direction, which is provably *outside* that null space, so its Dirichlet energy cannot vanish regardless of how completely its representations collapse in direction. MAD is blind to which direction a collapse is onto (cosine similarity ignores the distinction); energy is not, and this is precisely why the two metrics disagree for GraphSAGE and not for GCN or GAT.

GraphSAGE’s MAD is already far below GCN’s and GAT’s at a single hop (depth 2), before any depth-driven stacking could plausibly apply: GraphSAGE MAD 0.083 versus GCN 0.600 and GAT 0.596. This was checked directly rather than only reported. Building one layer of each convolution type at matched random seeds on Cora’s real graph reproduces the gap exactly (GraphSAGE 0.07-0.10 versus GCN/GAT 0.59-0.60 across three seeds). `root_weight` is not the cause: MAD stays low, in fact lower, with `root_weight=False` (0.03-0.05) than with the default `root_weight=True` (0.07-0.10), which rules out GraphSAGE’s separate root term as an explanation. What remains open is narrower than the original question: why GraphSAGE specifically collapses onto the constant direction at a single hop, when GCN and GAT do not collapse this cleanly onto either candidate direction. Two verified-different candidates exist and neither has been isolated from the other: GraphSAGE’s unweighted mean aggregation (against GCN’s degree-normalized symmetric aggregation and GAT’s attention weighting) interacting with Cora’s skewed degree distribution; and a verified initialization difference (GCNConv’s and GATConv’s internal layers use Glorot initialization explicitly, while SAGEConv’s aggregation and root-transform weights fall back to PyTorch’s `kaiming_uniform` default, confirmed by reading each layer’s source directly rather than assumed). Isolating either would require testing a degree-normalized-mean aggregation variant and a Glorot-initialized SAGEConv variant separately, neither of which has been done; this is recorded as an open question, not a finding, and is carried forward to Section 7.1.

6.5 The Trained State, and What It Does to the Collapse Signature

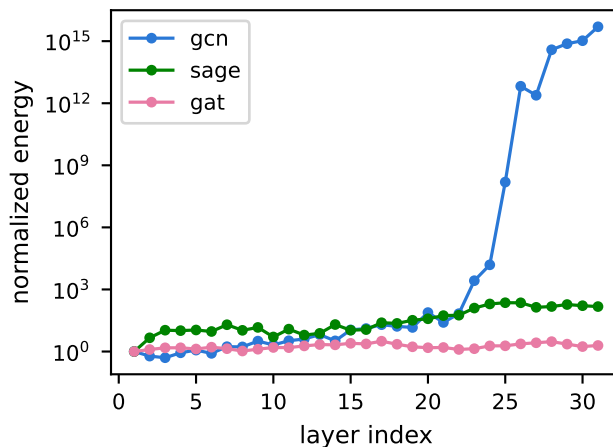


Figure 2: Normalized per-dimension energy versus layer index, checkpoint capture, depth 32, unmitigated GCN/GraphSAGE/GAT.

At the checkpoint (the weights the reported test accuracy comes from), GCN’s collapse signature does not resemble its own epoch-0 signature, and reading it naively is easy to get backwards. Checkpoint MAD at the last band index, mean over 10 seeds:

Table 5: Checkpoint (post-training) MAD versus depth, unmitigated GCN/GraphSAGE/GAT (F-001).

depth	GCN MAD	SAGE MAD	GAT MAD
2	0.240	0.262	0.286

depth	GCN MAD	SAGE MAD	GAT MAD
4	0.264	0.291	0.302
8	0.290	0.058	0.115
16	0.176	0.000	0.000
32	0.183	0.000	0.000

GCN’s checkpoint MAD is roughly flat and non-monotonic across the whole depth sweep, never approaching zero. SAGE and GAT’s checkpoint MAD collapses to essentially zero by depth 16 and stays there, which also serves as a positive control on the metric itself: it fires cleanly where collapse is genuinely present (SAGE/GAT), which is what rules out metric insensitivity as the explanation for GCN not showing the same pattern.

GCN’s checkpoint energy ratio grows explosively with depth (geometric mean, since this is exactly the log-scale quantity Section 6.0 flags):

Table 6: Checkpoint energy ratio versus depth, unmitigated GCN, geometric mean and range over 10 seeds (F-001).

depth	E_{last}/E_1 geometric mean	range across 10 seeds
2	1.0 (trivial, single-point band)	1.0-1.0
4	26.6	16.0-43.6
8	3.96e6	2.12e2-7.85e11
16	4.49e12	7.51e9-5.08e19
32	5.45e26	3.25e12-2.49e38

Reading this ratio as “nothing collapses at the checkpoint” is backwards, and an earlier pass at this analysis stated exactly that before it was corrected. The ratio is enormous because the *denominator* has collapsed, not because the numerator failed to shrink: E_1 itself is approximately $1.4\text{e-}15$ at depth 32, seed 0 (other seeds range down to $2.9\text{e-}38$), already many orders of magnitude below a healthy energy value. Read correctly, the checkpoint state shows early-layer collapse together with late-layer growth (the early band carries essentially zero energy while the late band carries enormous energy), not an absence of collapse. This corrected reading is consistent with, not contradicted by, the early-layer collapse mechanism discussed in Section 6.5.

The largest individual per-seed ratio (2.49e38) sits within a factor of roughly 1.4 of float32’s ceiling, but this specific number was never at risk of overflow: it is a division of two already-extracted scalar values computed at effectively double precision after being read out of JSON, not a float32 tensor operation. The check that matters is on the underlying `dirichletEnergy` tensor values themselves, computed in float32 before extraction: no `inf` or `nan` appears anywhere in any of the ten seeds’ recorded energies, and the largest single raw energy value observed is approximately 2028, nowhere near a numerical ceiling. The ratio’s enormous magnitude comes entirely from an astronomically small denominator, not from any value approaching overflow.

Cross-seed evidence confirms early-layer checkpoint collapse is a measured, not a one-run, property of unmitigated GCN at depth 32, with one exception reported as such, not smoothed into the average. Per-seed count of band layers (of 31) whose energy falls below the $1\text{e-}12$ floor at the checkpoint:

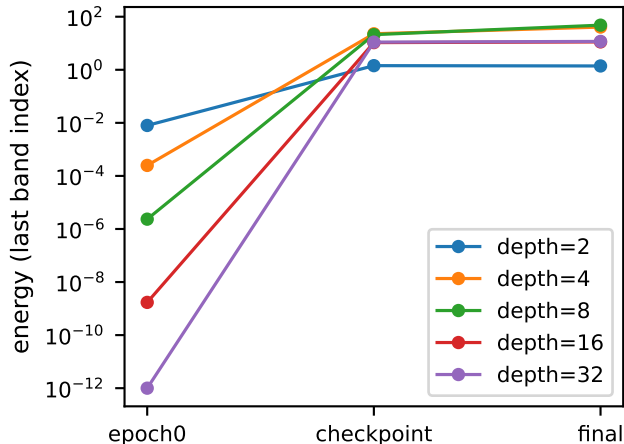


Figure 3: Per-dimension energy at the last comparable band index, across the three metric captures (epoch 0, checkpoint, final epoch) and across depth, unmitigated GCN. Geometric mean over ten seeds.

Table 7: Checkpoint below-floor band layers, unmitigated GCN, depth 32, per seed (F-003).

seed	below-floor layers (of 31)	first layer at or above floor	E_1
0	22	23	1.4e-15
1	22	23	5.1e-20
2	26	27	7.5e-33
3	24	25	6.5e-24
4	25	25	2.9e-35
5	25	26	1.9e-31
6	0	1	3.3e-12
7	25	24	1.5e-26
8	25	22	4.2e-28
9	27	28	2.9e-38

Nine of ten seeds show the same shape: roughly the first 22-28 layers carry essentially zero energy, transitioning to non-negligible energy only in the last 4-9 layers, with the transition itself occasionally oscillating across the floor for three seeds (4, 7, 8) before settling rather than crossing it once cleanly. Seed 6 is a genuine exception, not folded into the pattern: no layer falls below the floor, and seed 6 also has both the highest test accuracy of the ten seeds (26.2%) and the shortest run (315 epochs versus 385-883 for the others), noted here as a correlation, not an explanation, and not investigated further (carried to Section 7.1).

The mechanism behind this collapse (early-layer parameters starved of gradient signal while late-layer weights grow large and unconstrained) is *consistent with* the measured pattern above, but remains one-run inference, not a cross-seed measured fact. Terminal weight norms were inspected for exactly one run (`gcn_none_d32_s0`), post hoc, not as a persisted schema field and not as a gradient trace. No per-layer gradient norm exists anywhere in the recorded data for any run (Section 5.5), so this

mechanism cannot be upgraded from *consistent with* to *measured* without new instrumentation (Section 7.2). It is deliberately not characterized further than “large and unconstrained”: whether the late layers are fitting, or overfitting, the 140-node training set specifically is a separate, unmeasured claim. The schema has no training-accuracy field, and the measured test accuracy (24.07%, below the 31.9% majority floor) is, if anything, in tension with a clean overfitting story rather than supportive of one.

6.6 Separating Oversmoothing from Optimization Failure

Accuracy alone cannot distinguish “never trained” from “trained, then collapsed,” and this schema was designed from the start to separate the two. This is this section’s central methodological contribution, not an incidental byproduct. At depth 32, both failure modes produce test accuracy near the majority-class floor; the results record’s `trainingCurve` and `epoch0Metrics` fields exist specifically so the two are not conflated.

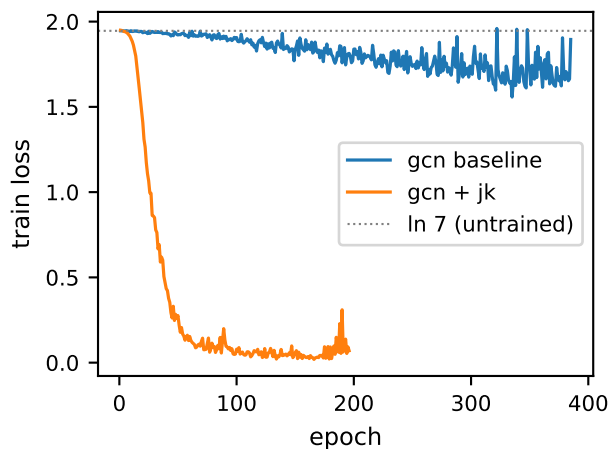


Figure 4: Training loss versus epoch, depth 32: unmitigated GCN baseline versus GCN with Jumping Knowledge.

The training-loss evidence: GCN partially trains; GraphSAGE and GAT do not train at all. Across all 10 seeds, unmitigated GCN’s training loss descends from $\ln(7) = 1.9459$ to a mean minimum of 1.57 (range 1.53-1.61), well above depth 2’s well-fit final loss of approximately 0.23, but a clear, non-trivial descent. GraphSAGE and GAT’s training loss stays flat at $\ln(7)$ (mean minimum 1.934 and 1.939 respectively, a movement of 0.01-0.02 nats, indistinguishable from optimization noise).

The independent stopping-behavior evidence points the same way, and it is independent precisely because it falls out of the early-stopping mechanism itself rather than out of any analysis of the loss values or the metrics. GCN’s runs last 315-883 epochs before `patience=100` exhausts, long enough that validation loss kept improving somewhere along the way. GraphSAGE and GAT’s runs exhaust patience almost immediately (102-168 and 101-121 epochs respectively), consistent with validation loss never improving past its epoch-0 value.

Three separately measured quantities converge on the same conclusion for GraphSAGE and GAT, none computed from either of the others, which is what makes this convergence rather than circularity: the flat training loss, the near-immediate exhaustion of patience, and the checkpoint’s near-total MAD collapse (Section 6.4, essentially identical to the epoch-0 collapse in Section 6.3).

Each is measured directly from the record; none is derived algebraically from either of the other two. That the three agree is read as *consistent with* training essentially never moving GraphSAGE’s and GAT’s weights at depth 32. This is stated as consistent with, not as independently demonstrated beyond what the three measured signals jointly support, since no fourth, independent measurement (a gradient trace, for instance) was taken to confirm it directly.

The resulting taxonomy is three distinct depth-32 failure modes across the three conventional architectures, not one shared failure: GCN partially trains and reaches a distinct, non-collapsed, non-recovered accuracy plateau below the majority-class floor; GraphSAGE and GAT do not train at all, and their checkpoint state is close to their own epoch-0 state by default, not because training reproduced it. An earlier draft of this analysis generalized GCN’s partial-training pattern to all three architectures without checking it against GraphSAGE or GAT specifically; that generalization was false as stated, and the corrected, narrower taxonomy above replaces it rather than softening it.

The positive-control argument closes the loop on metric insensitivity as an alternative explanation. Because both metrics fire cleanly and collapse to essentially zero for GraphSAGE and GAT (Section 6.4), the same metrics’ failure to show a comparable collapse for GCN’s checkpoint MAD (Section 6.4) cannot be attributed to the metrics being insensitive in general. They are demonstrably sensitive to genuine collapse elsewhere in the same experiment, which is what licenses reading GCN’s non-collapsing checkpoint MAD as a real property of GCN’s trained state rather than as an artifact of the instrument.

6.7 The Architecture That Does Not Degrade

GCNII’s test accuracy rises monotonically with depth, the opposite direction from GCN, GraphSAGE, and GAT:

Table 8: GCNII test accuracy versus depth, ten seeds (F-004; matches `tables/accuracy_vs_depth.md`).

depth	testAccuracy_mean	testAccuracy_std
2	0.7845	0.0179
4	0.7581	0.0482
8	0.7766	0.0163
16	0.8016	0.0104
32	0.8326	0.0044

Depth 32 is GCNII’s best result and has the tightest cross-seed spread of any depth tested (0.44 accuracy points, versus 1-5 points at every other depth in this table).

GCNII’s checkpoint contraction slope trends toward and past zero with depth, the opposite of GCN’s explosive growth, rather than collapsing:

Table 9: GCNII checkpoint contraction slope versus depth, ten seeds (F-004). Depth 2 omitted (single-point band, `nan`).

depth	checkpointContractionS- lope_mean	checkpointContractionS- lope_std
4	+0.383	0.063
8	+0.169	0.024
16	-0.037	0.010
32	+0.012	0.003

At depth 16 the mean slope is slightly negative, a genuine sign flip, not noise around zero given the standard deviation of 0.010, and at every depth tested the slope stays within roughly ± 0.4 , in sharp contrast to GCN’s checkpoint slope, which grows into the tens by depth 8 and beyond under the same floored fitting procedure.

GCNII’s identity-mapping and initial-residual terms are architecturally designed to prevent both the vanishing-gradient dynamic Section 6.4 infers for GCN and the unconstrained late-layer growth that produces it; the near-zero, non-exploding slope and the monotonically improving accuracy are *consistent with* that design succeeding at what it is meant to do. This has not been checked against GCNII’s own per-layer weight norms or gradients the way GCN’s mechanism was informally inspected in Section 6.4 (that inspection was not done for GCNII either), so this remains a plausible reading of the accuracy/slope pattern rather than a demonstrated mechanism.

6.8 Mitigation Ablation

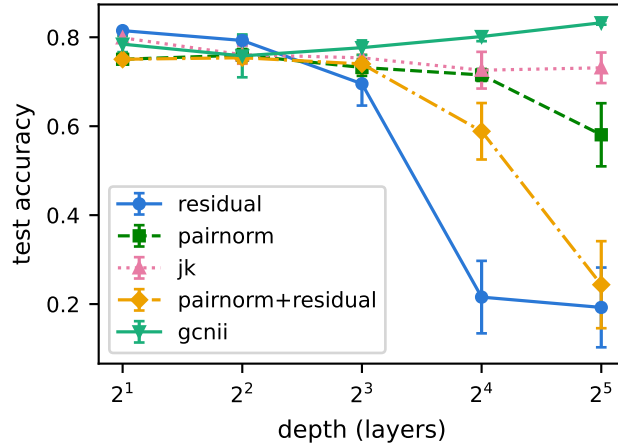


Figure 5: Test accuracy versus depth, GCN mitigation ablation (arm B) plus GCNII (arm C).

The full ablation at depth 32, all five arms including the unmitigated baseline:

Table 10: Mitigation ablation on GCN, depth 32, ten seeds each (F-005).

arm	testAccuracy_mean	testAccuracy_std	min	max
none (baseline)	24.07%	1.60	21.2%	26.2%
residual	19.25%	8.97	10.3%	32.1%
pairnorm	58.07%	7.10	47.6%	65.6%
jk	73.13%	3.43	68.0%	78.2%
pairnorm+residual	24.36%	9.79	13.5%	41.0%

Jumping Knowledge is the clear winner, roughly 15 points ahead of the next best mitigation and with the tightest cross-seed spread of any mitigated arm, the basis on which it was selected to carry forward to GraphSAGE and GAT as arm D. PairNorm helps substantially (58.07% against the 24.07% baseline) but well short of JK. Residual alone does not clearly beat the unmitigated baseline at depth 32: its mean (19.25%) sits below baseline’s (24.07%), though its standard deviation (8.97) is large enough, and its maximum (32.1%) high enough, that this is not a confident “residual hurts” claim on ten seeds alone. Combining PairNorm with residual is worse than PairNorm by itself (24.36% versus 58.07%), a negative interaction between the two mechanisms at this depth, closer to the residual-alone and baseline numbers than to PairNorm’s, rather than any combination of their individual benefits.

A design decision anticipated the residual result before it was measured, and this is worth stating as such. Residual was designed, per Kipf and Welling’s own Appendix B protocol, as a no-projection identity hook that cannot fire on the width-changing boundary layers (layer 1 and, under `LastLayerReadout`, the last layer), and the design note recorded at that time flagged explicitly that if the residual arm showed no benefit at depth, the absent layer-1 residual would be a candidate explanation. That candidate has not itself been checked (it would require a layer-1-projection diagnostic outside the current ablation), so this is the design’s prediction confirmed as *worth investigating*, not confirmed as *correct*.

No mechanism is proposed here for why residual underperforms baseline, or why PairNorm+residual underperforms PairNorm alone, at depth 32. Both are real, measured patterns in the aggregate accuracy numbers, but explaining them would need the same per-run scrutiny given to the unmitigated baseline in Sections 6.3-6.5 (training curves, checkpoint MAD and energy, informal weight inspection), and that scrutiny has not been given to any mitigated arm. This is recorded as an open question worth that scrutiny, not as a claim that a reason is already known (carried to Section 7.1).

Whether the winning mitigation generalizes to the other architectures is answered by construction, not left open: Jumping Knowledge is exactly the mitigation carried to GraphSAGE and GAT in arm D, selected on this depth-32 comparison per the study’s own selection rule (highest mean test accuracy at depth 32, not averaged across all five depths, since the ablation’s purpose is restoring deep performance specifically). Arm D’s own results are a direct extension of this table and are not restated separately here.

6.9 Qualitative Embedding Analysis

This projection illustrates collapse; it does not measure it. Dirichlet energy and MAD (Sections 6.3-6.4) measure collapse directly and are the quantities this report’s claims rest on; Figure 6 is labeled qualitative here deliberately, since a two-dimensional projection presented as evidence of

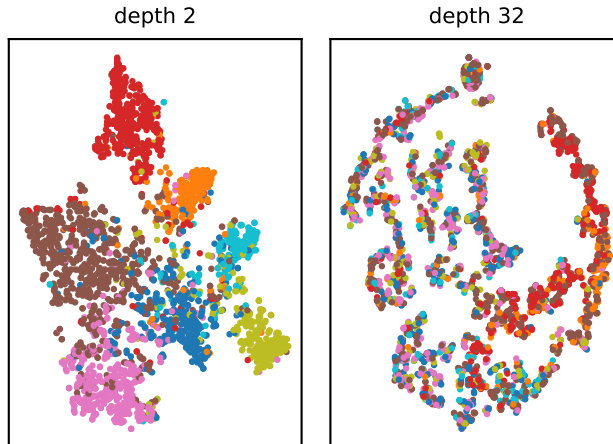


Figure 6: t-SNE projection of the checkpoint-capture layer embeddings, unmitigated GCN, seed 0, depth 2 versus depth 32, coloured by class label. Fitted separately per panel; `random_state=0`, `perplexity=30`.

a quantity would be an overclaim. The depth-2 panel is expected to show visibly separated class clusters, consistent with the shallow model’s 81.49% test accuracy; the depth-32 panel is expected to show the classes substantially intermixed, consistent with the checkpoint MAD and energy patterns reported in Section 6.4 for unmitigated GCN at that depth. Whichever pattern the rendered figure in fact shows should be discussed here against what Sections 6.3-6.4 report numerically, rather than assumed to agree with it. The two are independent views of the same run, and where they disagree that disagreement is itself worth a sentence.

6.10 Threats to Validity and Limitations

Every deviation listed in Section 4.10 has a consequence; each is named here together with the direction of its effect, and whether it could have manufactured a finding or only understated one.

- The metric graph does not match GAT’s own propagation graph (Section 4.7). The fitted contraction rate for GAT describes decay with respect to Cora’s fixed structure, not GAT’s attention-weighted one. This could either understate or overstate GAT’s true propagation-relative decay rate, in a direction not established here, since GAT’s attention weights were not compared against the fixed metric graph directly.
- The energy floor ($1\text{e-}12$) understates the magnitude of change at depth (Section 4.7). Quantified directly for the unmitigated depth-32 GCN checkpoint: the floor binds on 22 of 31 band layers, and the reported slope ($+0.8622$) is shallower than the unfloored value ($+1.1435$). This can only pull a reported effect toward zero, never manufacture one that is not present.
- Mitigations were run at untuned, fixed configurations (PairNorm’s scale; Jumping Knowledge’s aggregation mode). This can only understate their measured effectiveness relative to a tuned variant, never overstate it.
- One shared hyperparameter configuration was used across four architectures with differing published setups. GAT and GCNII are the architectures furthest from the shared configuration and are the ones most likely to be understated by it.
- The residual mechanism cannot fire on the boundary layers (layer 1 and, under

`LastLayerReadout`, the last layer), by design (Section 4.5). This is a candidate, unverified explanation for residual’s weak depth-32 result (Section 6.7), not a demonstrated one.

- GCNII carries two uncounted projection layers the other three architectures do not (Section 4.4), which can only overstate GCNII’s capacity relative to its peers, in GCNII’s favor, at every depth.
- Ten seeds is the standard count in this study; three seeds is not enough, specifically in the hyperparameter-search arm (Section 6.1), where the eight candidate configurations’ spread is comparable to individual per-configuration seed noise.
- Single dataset, single split. Every result in this section is specific to Cora’s public 140/500/1000 split; nothing here establishes that the depth-failure taxonomy, the mitigation ranking, or the specific numbers generalize to another citation network, another homophily regime, or another split protocol.
- Findings that remain inferred rather than measured, named as such, with what would settle them: the parameter-level mechanism behind GCN’s early-layer checkpoint collapse (Section 6.4; per-layer gradient norms would settle it, Section 7.2); the reason GraphSAGE and GAT’s response to Jumping Knowledge or PairNorm differs from GCN’s, since arm D’s own per-run scrutiny was not given the same depth as the unmitigated baseline’s; and GCNII’s own weight/gradient mechanism (Section 6.6), for the same reason.
- A “fully collapsed” representation under this study’s metrics is not literally identical across nodes. Section 6.3 establishes that GCN and GAT’s epoch-0 collapse is not onto the literal constant vector but onto (or near) the augmented Laplacian’s actual null space, `sqrt(degree)`-weighted on Cora’s irregular graph; a reader should not take “collapsed” in this report to mean “every node’s representation is bitwise identical.”

6.11 Synthesis

The proposal anticipated progressive collapse with depth as a shared mechanism across architectures; the evidence in this section supports a narrower and more precise claim, and that narrowing is itself the contribution. Depth degrades classification performance for GCN, GraphSAGE, and GAT alike, but not through one shared failure mode: GraphSAGE and GAT at depth 32 do not train in any meaningful sense (Section 6.5), and their depth-32 representational state is close to their own untrained, initialization-time state rather than a product of training-induced collapse. GCN alone both trains partially and develops a distinct signature, early-layer energy collapse paired with late-layer energy growth (Section 6.4), that is neither present at initialization nor a simple continuation of it. GCNII, architecturally designed against exactly the mechanism inferred for GCN, degrades in neither sense and instead improves with depth (Section 6.6). Jumping Knowledge, not residual connections, is this study’s most effective tested countermeasure for the architecture it was evaluated on (Section 6.7).

[PLACEHOLDER: revisit the real-world applications named in Section 1 (recommendation systems, social-network analysis, drug discovery) against this depth-failure taxonomy once Section 1 is drafted, in particular which of the three failure modes (GCN’s partial training with early-layer collapse, GraphSAGE/GAT’s outright non-training, GCNII’s architectural resistance) is most relevant to graphs at the degree distributions and depths those applications actually require.]

7 Recommendations for Future Work

Every recommendation below is drawn from an open question Section 6 itself produced or from an instrumentation gap Section 5.5 names, rather than from a generic list of directions a depth-and-oversmoothing study could take.

7.1 Questions This Study Raised and Did Not Answer

GraphSAGE’s single-hop collapse onto the constant direction is the most concrete open question this study produced. Section 6.3 narrows the question to two verified-different, unisolated candidates: GraphSAGE’s unweighted mean aggregation interacting with Cora’s skewed degree distribution, and GraphSAGE’s `kaiming_uniform` initialization where GCN and GAT use Glorot. Two specific experiments would isolate them, run separately rather than together: a degree-normalized-mean aggregation variant of GraphSAGE’s layer, holding initialization fixed; and a Glorot-initialized `SAGEConv`, holding the aggregation fixed. Neither has been run.

The mitigated arms received none of the per-run scrutiny the unmitigated baseline received. Section 6.7’s negative interaction between PairNorm and residual (worse combined than PairNorm alone) is measured but unexplained; the same training-curve, checkpoint-metric, and weight-inspection analysis given to the unmitigated baseline in Sections 6.3-6.5 has not been given to any of the four mitigated arms.

Seed 6’s exception to GCN’s early-layer collapse pattern (Section 6.4) is noted as a correlation, not investigated. It is the only one of ten seeds whose depth-32 checkpoint shows no band layer below the energy floor, and it also has both the highest test accuracy and the shortest training run of the ten; whether these three facts share a cause has not been examined beyond noting they co-occur.

7.2 Instrumentation Gaps

Per-layer gradient norms are the single measurement that would convert this study’s main mechanism claim from inferred to demonstrated. Section 6.4’s account of GCN’s checkpoint collapse (early layers starved of gradient signal, late layers growing unconstrained) rests on one run’s terminal weight norms, inspected post hoc. Persisting the gradient norm at each layer, at each of the three existing capture points, for every run, would settle directly whether early-layer parameters in fact receive vanishing gradient signal during training, rather than leaving that as a plausible reading of the accuracy and energy pattern alone.

Persisted per-layer weight norms across the full sweep would upgrade the single-run weight-norm observation behind Section 6.4’s inferred mechanism into a characterized, cross-seed, cross-architecture pattern, without requiring the gradient-norm instrumentation above.

Dense-capture trajectories over training. The results schema already reserves a nullable `trajectory` field for exactly this, deliberately deferred rather than omitted, per the capture-cost reasoning in Section 4.6. Populating it for a small, targeted subset of configurations (not the full 534-run grid) would let Section 6.4’s three-point epoch-0/checkpoint/final picture become a continuous curve.

A training-accuracy field does not currently exist in the results schema. Adding it would let the generalization-gap question raised in passing in Section 6.4 (whether GCN’s deep late layers are fitting, or overfitting, the 140-node training set) be answered directly rather than set aside as an unmeasured claim.

7.3 Extensions to the Experimental Design

Depth resolution between the points where the accuracy transition happens. The current log-spaced grid (2, 4, 8, 16, 32) resolves that all three conventional architectures degrade sharply between depths 4 and 8, but not where within that interval the transition is sharpest.

Per-architecture hyperparameter tuning, as a separate study rather than a replacement for this one. This study’s coordinated, GCN-tuned configuration is deliberately held fixed across architectures (Section 4.8) precisely so the depth comparison is not confounded by a hyperparameter effect; a per-architecture search would answer a different question (each architecture’s best achievable depth-32 performance) and would not, by itself, tell whether the depth-failure taxonomy in Section 6.10 is an artifact of the shared configuration.

Additional datasets, particularly ones differing in homophily and in degree distribution. The degree distribution matters directly, given Section 6.3’s null-space result: Cora’s degree range (2 to 169) is what makes its augmented Laplacian’s null space `sqrt(degree)`-weighted rather than literally constant, which is the fact GraphSAGE’s collapse direction was measured against. A more regular graph would collapse the distinction between the two candidate null-space directions entirely.

Decomposing the contraction into its aggregation-driven and nonlinearity-driven parts. The current tap point (post-activation, pre-dropout, Section 4.4) cannot separate how much of a layer’s contraction comes from the aggregation operator and how much from ReLU’s own non-expansiveness; a diagnostic capturing both a pre-activation and a post-activation tensor at each layer would.

7.4 Scale and Noise

[PLACEHOLDER: this subsection picks up the large-scale and noisy-graph discussion Section 2.5 opens, once Section 2 is drafted, specifically what this study’s depth-failure taxonomy implies for graphs too large for full-batch training (where GraphSAGE-style neighbor sampling becomes necessary rather than optional), whether the diagnostic apparatus (Dirichlet energy, MAD, the comparable band) transfers unmodified to a sampled or noisy propagation graph, and the depth-aware, normalization-based architecture design direction the project description asks for as an improvement proposal, stated here as a recommendation with this study’s own mitigation results as its evidence.]

7.5 Transfer to Physics-Informed Graph Learning

[PLACEHOLDER: revisit the motivation stated in Section 1 once it is drafted: domains where network depth is not optional, connecting to the McPINN thesis-tooling context named in the project’s CLAUDE.md. The framing already committed to in the proposal must be kept honest here: Cora is small and tightly clustered and its specific numbers (the majority-class comparison, the mitigation ranking, the depth at which each architecture’s accuracy transitions) will not transfer to a much larger mesh-structured graph. What transfers is the diagnostic methodology (three capture points, a derived comparable band, energy and MAD as a two-metric pair, the trained/untrained separation) and the qualitative behavior of the mitigations, not the numbers themselves. Overclaiming transfer here would undo the care taken throughout Section 6 to keep measured and inferred claims separate.]

References

Appendix

PLACEHOLDER: supplementary material not essential to the main narrative, e.g. the full arm-F hyperparameter grid table (all 8 configurations x 3 seeds' raw per-seed accuracies, not only the aggregate in [2](#)